



NAME: TARUNIKA S

COURSE : DESIGN AND ANALYSIS OF ALGORITHM

COURSE CODE: CSA 0618

REGNO: 192511024

TOPIC 1

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω : $O(1)$, O : $O(n)$, Θ : $\Theta(n)$
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

ANSWER

PROGRAM

```
arr = [10, 25, 30, 45, 50]
```

```
key = 30
```

```
found = False
```

```
for i in range(len(arr)):
    if arr[i] == key:
        print("Element found at index:", i)
        found = True
        break
```

```
if not found:
    print("Element not found")
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 arr = [10, 25, 30, 45, 50] 2 key = 30 3 4 found = False 5 6 for i in range(len(arr)): 7 if arr[i] == key: 8 print("Element found at index:", i) 9 found = True 10 break 11 12 if not found: 13 print("Element not found")</pre>			Element found at index: 2

Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** $\Omega: O(1)$, $O: O(\log_{10} n)$, $\Theta: \Theta(\log_{10} n)$
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

ANSWER

PROGRAM

```
arr = [5, 10, 15, 20, 25]
key = 20
```

```
low = 0
```

```
high = len(arr) - 1
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == key:
```

```
        print("Element found at index:", mid)
```

```
        break
```

```
    elif arr[mid] < key:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
else:
```

```
    print("Element not found")
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 arr = [5, 10, 15, 20, 25] 2 key = 20 3 4 low = 0 5 high = len(arr) - 1 6 7 while low <= high: 8 mid = (low + high) // 2 9 10 if arr[mid] == key: 11 print("Element found at index:", mid) 12 break 13 elif arr[mid] < key: 14 low = mid + 1 15 else: 16 high = mid - 1 17 else: 18 print("Element not found")</pre>			Element found at index: 3

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** $n = 5$
- **Sample Output:** 120
- **Complexity:** Both $O(n)$
- **Explanation:** Highlights recursion depth vs iterative loops.

ANSWER

PROGARM

```
n = 5
```

```
fact = 1
```

```
for i in range(1, n + 1):
```

```
    fact = fact * i
```

```
print("Factorial:", fact)
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 n = 5 2 fact = 1 3 4 for i in range(1, n + 1): 5 fact = fact * i 6 7 print("Factorial:", fact)</pre>			Factorial: 120

Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** n = 6
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative $O(n)$, Recursive naïve $O(2^n)$, Recursive with memoization $O(n)$
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

ANSWER

PROGRAM

```
n = 6
```

```
a = 0
```

```
b = 1
```

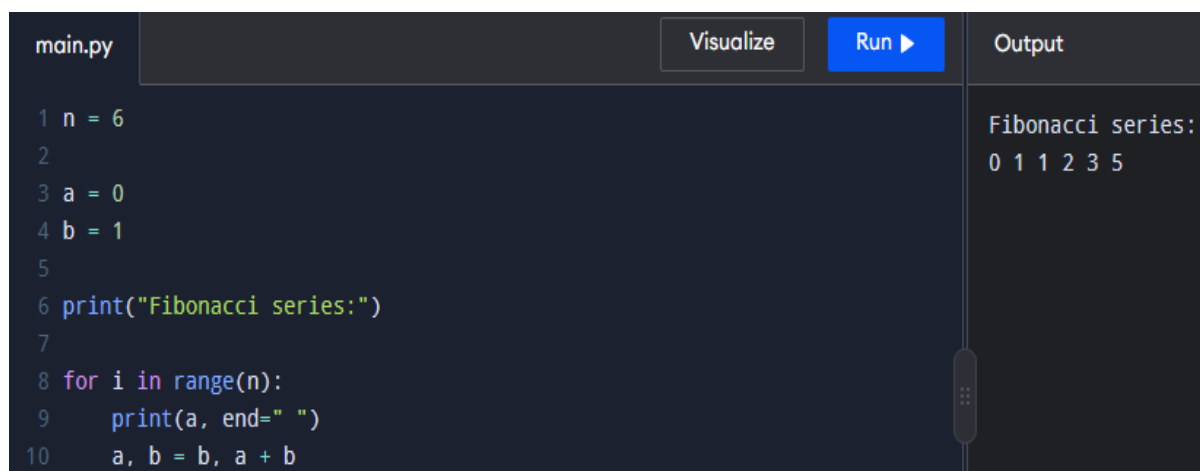
```
print("Fibonacci series:")
```

```
for i in range(n):
```

```
    print(a, end=" ")
```

```
    a, b = b, a + b
```

OUTPUT



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 n = 6
2
3 a = 0
4 b = 1
5
6 print("Fibonacci series:")
7
8 for i in range(n):
9     print(a, end=" ")
10    a, b = b, a + b
```

There are buttons for 'Visualize' and 'Run' (with a play icon). The 'Output' panel on the right shows the result of running the code:

```
Fibonacci series:
0 1 1 2 3 5
```

Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** A=[[1,2],[3,4]], B=[[5,6],[7,8]]
- **Sample Output:** [[19,22],[43,50]]
- **Complexity:** $O(n^3)$
- **Explanation:** Shows cubic growth due to nested loops.

ANSWER

PROGRAM

```
A = [[1, 2],
      [3, 4]]

B = [[5, 6],
      [7, 8]]
result = [[0, 0],
          [0, 0]]

for i in range(2):
    for j in range(2):
        for k in range(2):
            result[i][j] += A[i][k] * B[k][j]

print("Result:")
for row in result:
    print(row)
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 A = [[1, 2], 2 [3, 4]] 3 4 B = [[5, 6], 5 [7, 8]] 6 7 result = [[0, 0], 8 [0, 0]] 9 10 for i in range(2): 11 for j in range(2): 12 for k in range(2): 13 result[i][j] += A[i][k] * B[k][j] 14 15 print("Result:") 16 for row in result: 17 print(row)</pre>			<pre>Result: [19, 22] [43, 50]</pre>

Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]
- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

ANSWER

PROGRAM

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
arr = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(arr))
```

OUTPUT



The screenshot shows a code editor with a file named 'main.py'. The code defines a 'merge_sort' function and applies it to the array [38, 27, 43, 3, 9, 82, 10]. The output of the program is displayed on the right side of the editor, showing the sorted array [3, 9, 10, 27, 38, 43, 82].

```
main.py Visualize Run ▶ Output
```

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     result = []
8     i = 0
9     j = 0
10    while i < len(left) and j < len(right):
11        if left[i] < right[j]:
12            result.append(left[i])
13            i += 1
14        else:
15            result.append(right[j])
16            j += 1
17    result.extend(left[i:])
18    result.extend(right[j:])
19    return result
20 arr = [38, 27, 43, 3, 9, 82, 10]
21 print(merge_sort(arr))
```

[3, 9, 10, 27, 38, 43, 82]

Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω : $O(n \log_{f0} n)$, O : $O(n^2)$, Θ : $\Theta(n \log_{f0} n)$
- **Explanation:** Shows how pivot choice affects performance.

ANSWER

PROGRAM

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[0]  
  
    left = [x for x in arr[1:] if x <= pivot]  
    right = [x for x in arr[1:] if x > pivot]  
  
    return quick_sort(left) + [pivot] + quick_sort(right)  
  
arr = [10, 7, 8, 9, 1, 5]  
  
print(quick_sort(arr))
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 def quick_sort(arr): 2 if len(arr) <= 1: 3 return arr 4 5 pivot = arr[0] 6 7 left = [x for x in arr[1:] if x <= pivot] 8 right = [x for x in arr[1:] if x > pivot] 9 10 return quick_sort(left) + [pivot] + quick_sort(right) 11 12 13 arr = [10, 7, 8, 9, 1, 5] 14 15 print(quick_sort(arr))</pre>			[1, 5, 7, 8, 9, 10]

Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** $O(2^n)$
- **Explanation:** Illustrates exponential growth in recursive calls.

ANSWER

PROGRAM

```
def hanoi(n, source, auxiliary, destination):  
    if n == 1:  
        print("Move disk 1 from", source, "to", destination)  
        return  
    hanoi(n - 1, source, destination, auxiliary)  
    print("Move disk", n, "from", source, "to", destination)  
  
    hanoi(n - 1, auxiliary, source, destination)  
n = 3  
  
hanoi(n, 'A', 'B', 'C')
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 def hanoi(n, source, auxiliary, destination): 2 if n == 1: 3 print("Move disk 1 from", source, "to", destination) 4 return 5 6 hanoi(n - 1, source, destination, auxiliary) 7 8 print("Move disk", n, "from", source, "to", destination) 9 10 hanoi(n - 1, auxiliary, source, destination) 11 12 13 n = 3 14 15 hanoi(n, 'A', 'B', 'C')</pre>			<pre>Move disk 1 from A to C Move disk 2 from A to B Move disk 1 from C to B Move disk 3 from A to C Move disk 1 from B to A Move disk 2 from B to C Move disk 1 from A to C</pre>

Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** $O(\log_{10} n)$
- **Explanation:** Shows efficient recursion with logarithmic complexity.

ANSWER

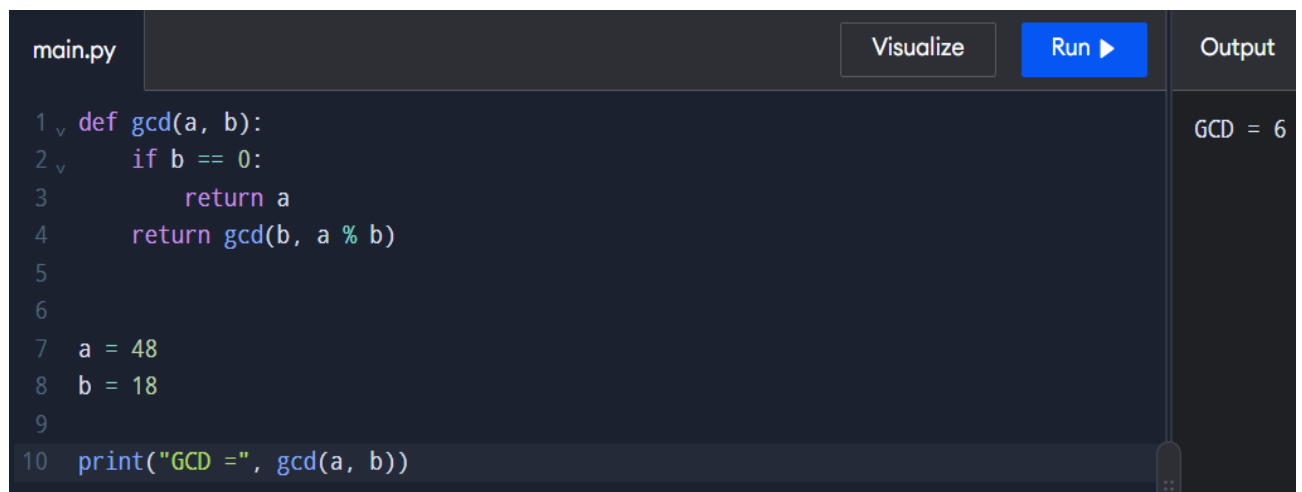
PROGRAM

```
def gcd(a, b):  
    if b == 0:  
        return a  
    return gcd(b, a % b)
```

```
a = 48  
b = 18
```

```
print("GCD =", gcd(a, b))
```

OUTPUT



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 def gcd(a, b):  
2     if b == 0:  
3         return a  
4     return gcd(b, a % b)  
5  
6  
7 a = 48  
8 b = 18  
9  
10 print("GCD =", gcd(a, b))
```

At the top right of the editor, there are buttons for 'Visualize' and 'Run'. The 'Run' button is highlighted in blue. To the right of the code editor, there is an 'Output' panel that displays the result: 'GCD = 6'.

Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** Ω : $O(n)$, O : $O(n^2)$, Θ : $\Theta(n^2)$
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

ANSWER

PROGRAM

```
arr = [12, 11, 13, 5, 6]

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print(arr)
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 arr = [12, 11, 13, 5, 6] 2 3 for i in range(1, len(arr)): 4 key = arr[i] 5 j = i - 1 6 7 while j >= 0 and arr[j] > key: 8 arr[j + 1] = arr[j] 9 j -= 1 10 11 arr[j + 1] = key 12 13 print(arr)</pre>			[5, 6, 11, 12, 13]

Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Consistent performance using heap structure.

ANSWER

PROGRAM

```
import heapq

arr = [4, 10, 3, 5, 1]

heapq.heapify(arr)

result = []

while arr:
    result.append(heapq.heappop(arr))

print(result)
```

OUTPUT



```
main.py Visualize Run ▶ Output
```

```
1 import heapq
2
3 arr = [4, 10, 3, 5, 1]
4
5 heapq.heapify(arr)
6
7 result = []
8
9 while arr:
10     result.append(heapq.heappop(arr))
11
12 print(result)
```

[1, 3, 4, 5, 10]

Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3
- **Complexity:** $O(n \log n)$
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

ANSWER

PROGRAM

```
arr = [2, 4, 1, 3, 5]

count = 0

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] > arr[j]:
            count += 1

print("Number of inversions =", count)
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 arr = [2, 4, 1, 3, 5] 2 3 count = 0 4 5 for i in range(len(arr)): 6 for j in range(i + 1, len(arr)): 7 if arr[i] > arr[j]: 8 count += 1 9 10 print("Number of inversions =", count)</pre>			Number of inversions = 3

Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7
- **Complexity:** Kadane's $O(n)$, Divide & Conquer $O(n \log n)$
- **Explanation:** Highlights efficiency differences between approaches.

ANSWER

PROGRAM

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]

max_sum = arr[0]
current_sum = arr[0]

for i in range(1, len(arr)):
    current_sum = max(arr[i], current_sum + arr[i])
    max_sum = max(max_sum, current_sum)

print("Maximum subarray sum =", max_sum)
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 arr = [-2, -3, 4, -1, -2, 1, 5, -3] 2 3 max_sum = arr[0] 4 current_sum = arr[0] 5 6 for i in range(1, len(arr)): 7 current_sum = max(arr[i], current_sum + arr[i]) 8 max_sum = max(max_sum, current_sum) 9 10 print("Maximum subarray sum =", max_sum)</pre>			Maximum subarray sum = 7

Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute x^n .
- **Sample Input:** $x=2, n=10$
- **Sample Output:** 1024
- **Complexity:** Iterative $O(n)$, Recursive fast power $O(\log_{10} n)$
- **Explanation:** Shows how recursion reduces complexity

ANSWER

PROGRAM

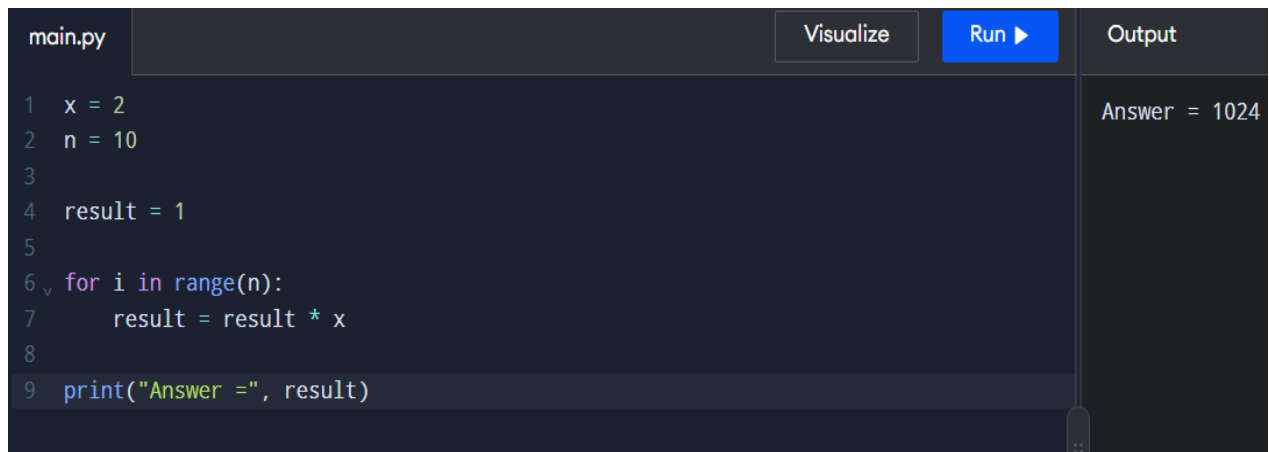
```
x = 2
n = 10

result = 1

for i in range(n):
    result = result * x

print("Answer =", result)
```

OUTPUT



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 x = 2
2 n = 10
3
4 result = 1
5
6 for i in range(n):
7     result = result * x
8
9 print("Answer =", result)
```

Buttons for 'Visualize' and 'Run' are visible above the code. The 'Run' button is highlighted in blue. To the right of the code editor, the output is displayed: 'Answer = 1024'.

Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** [(1,2), (4,5), (7,8), (3,1)]
- **Sample Output:** Closest pair (1,2)-(3,1), Distance = $\sqrt{5}$
- **Complexity:** Brute force $O(n^2)$, Optimized $O(n \log n)$
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.

ANSWER

PROGRAM

```
import math
points = [(1, 2), (4, 5), (7, 8), (3, 1)]

min_distance = float('inf')
closest_pair = None

for i in range(len(points)):
    for j in range(i + 1, len(points)):

        x1, y1 = points[i]
        x2, y2 = points[j]
        distance = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])

print("Closest pair:", closest_pair)
print("Distance:", min_distance)
```

OUTPUT

main.py	Visualize	Run ▶	Output
<pre>1 import math 2 3 points = [(1, 2), (4, 5), (7, 8), (3, 1)] 4 5 min_distance = float('inf') 6 closest_pair = None 7 8 for i in range(len(points)): 9 for j in range(i + 1, len(points)): 10 11 x1, y1 = points[i] 12 x2, y2 = points[j] 13 14 distance = math.sqrt((x2 - x1)**2 + (y2 - y1)**2) 15 16 if distance < min_distance: 17 min_distance = distance 18 closest_pair = (points[i], points[j]) 19 20 print("Closest pair:", closest_pair) 21 print("Distance:", min_distance)</pre>			<pre>Closest pair: ((1, 2), (3, 1)) Distance: 2.23606797749979</pre>

